

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA VẬT LÝ



TÔ QUANG HÂN

NHẬN DIỆN VÀ ĐẾM SỐ LƯỢNG PHƯƠNG TIỆN
GIAO THÔNG TRONG CÁC ĐIỀU KIỆN THỜI TIẾT
KHẮC NGHIỆT

Đồ án

Kỹ thuật điện tử và tin học

(Chương trình đào tạo chuẩn)

Hà Nội - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA VẬT LÝ



TÔ QUANG HÂN

NHẬN ĐIỆN VÀ ĐẾM SỐ LƯỢNG PHƯƠNG TIỆN
GIAO THÔNG TRONG CÁC ĐIỀU KIỆN THỜI TIẾT
KHẮC NGHIỆT

Đồ án

Kỹ thuật điện tử và tin học

(Chương trình đào tạo chuẩn)

Giảng viên hướng dẫn: TS. Nguyễn Tiến Cường

Hà Nội - 2026

Lời cảm ơn

Lời đầu tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy Nguyễn Tiến Cường, người đã luôn đồng hành, hướng dẫn và hỗ trợ em trong suốt quá trình học tập cũng như làm đồ án. Sự hỗ trợ chuyên môn và những kiến thức bổ trợ quý báu của thầy chính là kim chỉ nam giúp em vượt qua những khó khăn, từ đó hoàn thiện nghiên cứu một cách bài bản, khoa học và hiệu quả hơn.

Em cũng xin gửi lời cảm ơn đến gia đình, bạn bè, thầy cô, những người đã luôn quan tâm, động viên em trong suốt quá trình học tập.

Mặc dù đã cố gắng hoàn thiện đề tài một cách cẩn chu nhất, song do kiến thức và kinh nghiệm thực tế của bản thân còn hạn chế, quyền báo cáo chắc chắn không tránh khỏi những thiếu sót về mặt trình bày và thiết kế. Em rất mong nhận được những ý kiến đóng góp và nhận xét từ thầy để có thể tiếp tục hoàn thiện đồ án cũng như nâng cao kiến thức và kỹ năng của bản thân trong tương lai.

Cuối cùng, em xin kính chúc thầy luôn mạnh khỏe, hạnh phúc và gặt hái nhiều thành công trong công tác giảng dạy và nghiên cứu.

Xin chân thành cảm ơn!

Hà Nội, ngày 20 tháng 06 năm 2026

Sinh viên thực hiện

Tô Quang Hân

Mục lục

Lời cảm ơn	i
Danh sách hình vẽ	iv
Danh sách bảng	v
Danh sách tên viết tắt	vi
MỞ ĐẦU	1
1 TỔNG QUAN	3
1.1 Thị giác máy tính	3
1.2 Hệ thống giao thông thông minh	5
1.3 Những vấn đề của bài toán	6
1.4 Phương pháp giải quyết vấn đề	7
1.4.1 Phương pháp nhận diện vật thể	7
1.4.2 Phương pháp theo dõi đa đối tượng	7
1.4.3 Phương pháp đếm số lượng phương tiện giao thông	8
2 CƠ SỞ LÝ THUYẾT	9
2.1 Bài toán nhận diện vật thể	9
2.1.1 Mô hình hai giai đoạn	10
2.1.2 Mô hình một giai đoạn	11
2.1.3 Giới thiệu về mô hình YOLO	11
2.1.3.1 Nguyên lý cơ bản của YOLO	11
2.1.3.2 Các phiên bản YOLO	12
2.1.3.3 Kiến trúc YOLO 11	14
2.2 Bài toán theo dõi đa đối tượng	16
2.2.1 Bộ lọc Kalman	17
2.2.2 Thuật toán Hungarian	17
2.2.3 Nguyên lý hoạt động của Bytetrack	18
2.3 Các chỉ số đánh giá mô hình	19
2.3.1 Intersection over Union (IoU)	19
2.3.2 Precision (Độ chính xác) và Recall (Độ bao phủ)	20
2.3.3 F1-Score	21
2.3.4 Average Precision (AP) và Mean Average Precision (mAP)	21

3	THỰC NGHIỆM	23
3.1	Kiến trúc tổng quan của hệ thống	23
3.2	Huấn luyện mô hình	25
3.2.1	Chuẩn bị dữ liệu	25
3.2.1.1	Thông kê dữ liệu	26
3.2.1.2	Phân bố nhãn và thuộc tính dữ liệu	26
3.2.2	Môi trường và công cụ thực nghiệm	28
3.2.2.1	Cấu hình phần cứng	28
3.2.2.2	Môi trường phần mềm	28
3.2.3	Huấn luyện	28
3.3	Thiết lập Logic theo dõi và đếm xe	30
3.3.1	Thiết lập vùng đếm	30
3.3.2	Thiết lập logic đếm xe	31
4	KẾT QUẢ VÀ ĐÁNH GIÁ	33
4.1	Kết quả huấn luyện	33
4.1.1	Đánh giá hàm mất mát (Loss Curves)	33
4.1.2	Đánh giá các chỉ số hiệu suất	33
4.1.3	Đánh giá trên tập kiểm thử	34
4.2	Kết quả thực nghiệm	35
4.2.1	Đánh giá trong điều kiện thời tiết bình thường	35
4.2.2	Đánh giá trong điều kiện thời tiết khắc nghiệt	36
4.2.3	Đánh giá các yếu tố khác	36
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	38
5.1	Kết luận	38
5.2	Hướng phát triển trong tương lai	38
	Tài liệu tham khảo	40
A	Liệt kê source code	41
A.1	A.1 Cấu hình siêu tham số huấn luyện mô hình	41
A.2	A.2 Trích xuất tọa độ tham chiếu	41
A.3	A.3 Logic đếm phương tiện	42

Danh sách hình vẽ

1.1	Computer Vision.	4
1.2	Hệ thống giao thông thông minh.	5
2.1	Mô hình một giai đoạn và mô hình hai giai đoạn.	10
2.2	Nguyên lý cơ bản của YOLO.	12
2.3	Biểu đồ so sánh độ chính xác (COCO <i>mAP</i>) và độ trễ (Latency) giữa các thế hệ YOLO.	14
2.4	Kiến trúc YOLO11.	14
2.5	Intersection over Union.	20
3.1	Sơ đồ kiến trúc tổng quan của hệ thống	24
3.2	Bộ dữ liệu TSBOW.	25
3.3	Thống kê dữ liệu TSBOW.	26
3.4	Thuộc tính dữ liệu.	27
3.5	Thiết lập vùng đếm.	31
4.1	Kết quả huấn luyện.	33
4.2	Kết quả thực nghiệm trong điều kiện thời tiết bình thường.	35
4.3	Kết quả thực nghiệm trong các điều kiện thời tiết khắc nghiệt	37

Danh sách bảng

3.1	Cấu hình phân chia tập dữ liệu thực nghiệm	29
4.1	Kết quả đánh giá trên tập kiểm thử	34

Danh sách tên viết tắt

AI	Trí Tuệ Nhân Tạo
AP	Average Precision
C2PSA	Cross Stage Partial Spatial Attention
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSP	Cross Stage Partial
GPU	Graphics Processing Unit
IoU	Intersection over Union
mAP	mean Average Precision
NMS	Non Maximum Suppression
RAM	Random Access Memory
RoI	Region of Interest
SPPF	Spatial Pyramid Pooling Fast
TSBOW	Traffic Surveillance Benchmark for Occluded Vehicles Under Various Weather Conditions
VRAM	Video Random Access Memory
YOLO	You Only Look Once

MỞ ĐẦU

Trong những năm gần đây, Hệ thống giao thông thông minh (Intelligent Transportation Systems - ITS) đã trở thành một phần không thể thiếu trong việc điều tiết và quản lý cơ sở hạ tầng. Trong đó, hệ thống camera giám sát đóng vai trò quan trọng giúp thu thập và phân tích dữ liệu lưu lượng phương tiện.

Tuy nhiên, trong môi trường thực tế, các hệ thống camera giao thông phải đối mặt với những rào cản vật lý vô cùng khắc nghiệt. Thực tế cho thấy, các yếu tố môi trường như sương mù, mưa lớn, hay điều kiện thiếu sáng vào ban đêm làm giảm độ tương phản, làm mờ và gây nhiễu ảnh nghiêm trọng. Điều này dẫn đến tình trạng các thuật toán nhận diện truyền thống dễ bị mất dấu đối tượng, nhận diện sai hoặc bỏ sót phương tiện, làm sai lệch kết quả thống kê lưu lượng giao thông.

Xuất phát từ nhu cầu thực tiễn đó, em quyết định thực hiện đề tài: **"Nhận diện và đếm số lượng phương tiện giao thông trong các điều kiện thời tiết khắc nghiệt"**.

Để đảm bảo tính khoa học, về mặt dữ liệu huấn luyện, đề tài sử dụng bộ dữ liệu TSBOV (Traffic Surveillance Benchmark for Occluded Vehicles) chuyên biệt về giao thông trong điều kiện thời tiết xấu và bị che khuất. Về phương pháp triển khai mô hình, đề tài sử dụng mô hình mạng học sâu YOLO 11 tích hợp thuật toán theo dõi đa đối tượng Tracking, xây dựng logic đếm xe để chạy thử nghiệm trên các video giao thông thực tế.

Để làm rõ hơn về đề tài này, bài báo cáo được chia làm các nội dung như sau:

Chương 1: Tổng quan

Chương 2: Cơ sở lý thuyết

Chương 3: Thực nghiệm

Chương 4: Kết quả và đánh giá

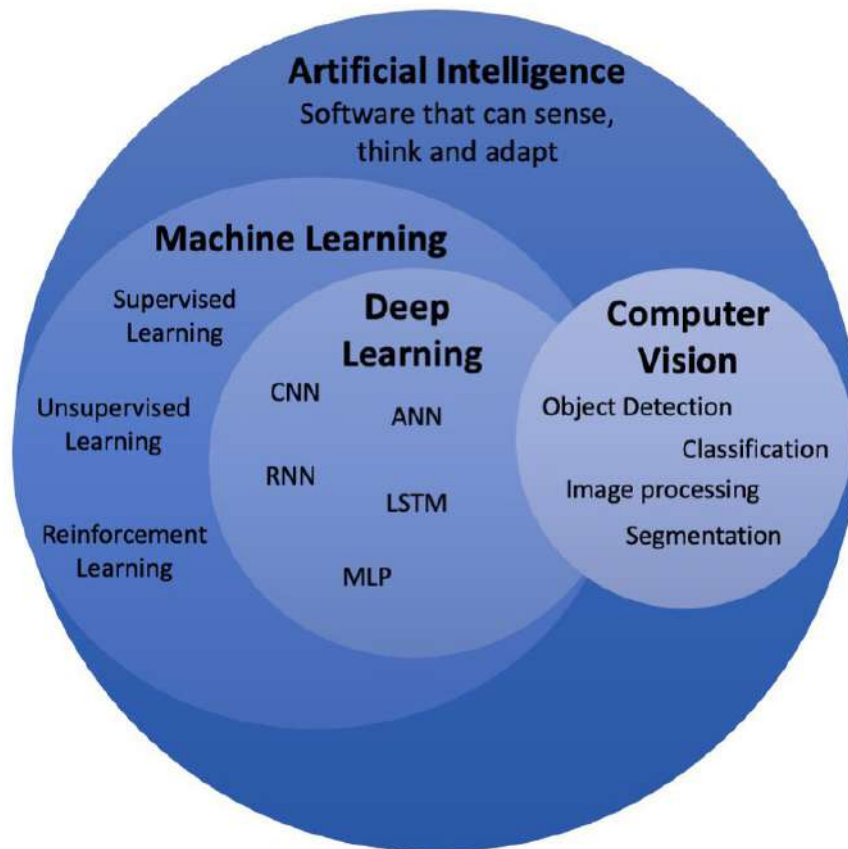
Chương 5: Kết luận và hướng phát triển

Chương 1 TỔNG QUAN

1.1 Thị giác máy tính

Thị giác máy tính (Computer Vision - CV) là một lĩnh vực giao thoa giữa Trí tuệ nhân tạo (AI) và Khoa học máy tính (Computer Science), tập trung vào việc nghiên cứu các kỹ thuật cho phép máy tính có khả năng thu nhận, xử lý và phân tích các hình ảnh hoặc video kỹ thuật số. Về mặt phân cấp hệ thống, CV không hoạt động độc lập mà có mối quan hệ giao thoa chặt chẽ với Học máy (Machine Learning) và Học sâu (Deep Learning) trong bức tranh tổng thể của Trí tuệ nhân tạo (AI).

Sự giao thoa được thể hiện rõ nét qua việc các mô hình CV hiện đại ngày càng dịch chuyển từ hệ thống xử lý ảnh dựa trên thuật toán thủ công cứng nhắc sang việc áp dụng các kiến trúc Học máy và Học sâu. Các kiến trúc này đóng vai trò như các bộ lọc tự động học mạnh mẽ, giúp trích xuất và tối ưu hóa các đặc trưng hình ảnh phức tạp mà các phương pháp truyền thống không thể chạm tới.



Hình 1.1: Computer Vision.

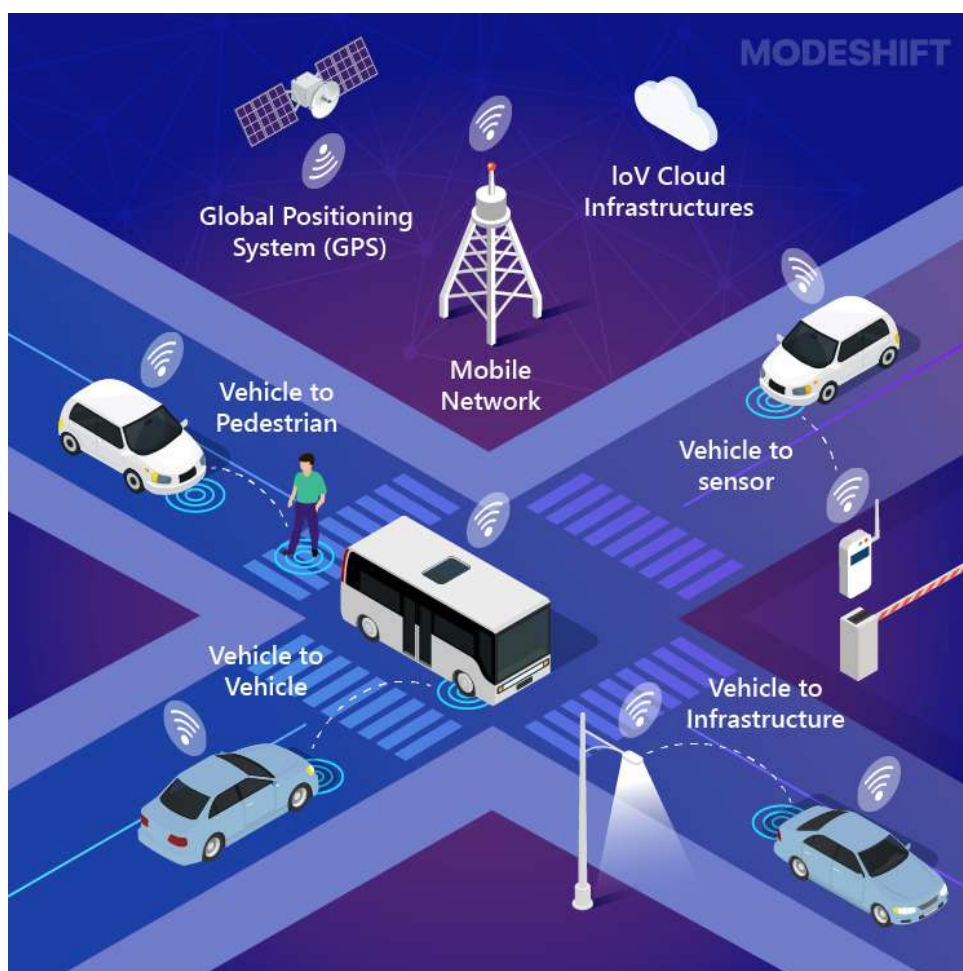
Các lĩnh vực chính của thị giác máy tính bao gồm:

- **Phân loại ảnh (Image Classification):** Xác định ảnh thuộc về một hoặc nhiều lớp.
- **Phát hiện đối tượng (Object Detection):** Xác định vị trí và loại đối tượng trong ảnh.
- **Nhận dạng khuôn mặt (Face Recognition):** Xác định hoặc xác thực danh tính dựa trên đặc điểm khuôn mặt.
- **Phân đoạn ảnh (Image Segmentation):** Chia ảnh thành các vùng tương ứng với từng đối tượng hoặc nền.
- **Theo dõi đối tượng (Object Tracking):** Xác định và theo dõi sự di chuyển của đối tượng qua các khung hình video.

- **Nhận dạng ký tự quang học (Optical Character Recognition - OCR):** Chuyển đổi văn bản trong ảnh thành văn bản số.
- **Ước lượng tư thế (Pose Estimation):** Xác định vị trí và tư thế của cơ thể người hoặc vật thể trong ảnh/video.

1.2 Hệ thống giao thông thông minh

Hệ thống giao thông thông minh (Intelligent Transportation Systems - ITS) là một hệ thống tích hợp toàn diện của các công nghệ viễn thông, điện toán đám mây, cảm biến IoT và Trí tuệ nhân tạo vào việc quản lý, điều hành mạng lưới giao thông. Mục tiêu cốt lõi của ITS là tối ưu hóa lưu lượng phương tiện, giảm thiểu ùn tắc, nâng cao an toàn và tự động hóa các quy trình giám sát (như phạt nguội, điều phối đèn tín hiệu).



Hình 1.2: Hệ thống giao thông thông minh.

ITS thu thập và xử lý dữ liệu từ nhiều nguồn như camera, đèn giao thông và phương tiện, trong đó, hệ thống camera giám sát là một trong những bộ phận quan trọng của toàn bộ mạng lưới, liên tục thu thập dữ liệu về lưu lượng dòng chảy giao thông trên đường.

1.3 Những vấn đề của bài toán

Mặc dù các hệ thống nhận diện phương tiện giao thông trong điều kiện bình thường đã đạt được độ chính xác cao, nhưng khi triển khai thực tế trên các hệ thống camera giám sát giao thông, bài toán đối mặt với những thách thức rất lớn:

- **Điều kiện môi trường:** Các hiện tượng thời tiết khắc nghiệt như tuyết rơi, mưa lớn hay sương mù làm giảm nghiêm trọng độ tương phản và ranh giới của vật thể. Bên cạnh đó, điều kiện ánh sáng yếu vào ban đêm làm mất đi đặc trưng màu sắc, giảm khả năng phát hiện và nhận diện đối tượng.
- **Góc nhìn và sự biến thiên kích thước:** Hầu hết camera được lắp đặt ở vị trí cao, tạo ra góc nhìn từ trên xuống. Điều này làm thay đổi hình chiếu của xe (Ví dụ: một chiếc xe SUV nhìn từ trên cao có thể mang đặc trưng ngoại hình giống với một chiếc xe tải nhỏ). Đồng thời, do hiệu ứng xa gần của ống kính camera, các phương tiện ở xa hoặc vật thể nhỏ chỉ chiếm một lượng pixel rất nhỏ, đòi hỏi hệ thống phải có khả năng xử lý chênh lệch kích thước tốt.
- **Hiện tượng che khuất:** Trong luồng giao thông đông đúc, đối tượng có thể bị che khuất bởi các vật thể khác như cây cối, đèn giao thông hoặc các phương tiện giao thông có kích thước lớn hơn, dẫn đến sai sót trong nhận diện và theo dõi.

Để giải quyết các vấn đề này, việc ứng dụng các kỹ thuật tiên tiến trong xử lý hình ảnh, trí tuệ nhân tạo và đặc biệt là các mô hình học sâu hiện đại là yêu cầu cấp thiết để tối ưu hóa hiệu suất và độ chính xác của hệ thống.

1.4 Phương pháp giải quyết vấn đề

1.4.1 Phương pháp nhận diện vật thể

Khác với bài toán phân loại ảnh (Image Classification) truyền thống chỉ trả lời câu hỏi: "*Trong ảnh có gì?*", bài toán nhận diện vật thể (Object Detection) đòi hỏi hệ thống phải giải quyết đồng thời hai nhiệm vụ: (1) Phân loại chính xác nhãn của đối tượng (Class) và (2) xác định chính xác tọa độ vị trí của đối tượng đó trên mặt phẳng ảnh thông qua Hộp giới hạn (Bounding Box).

Trong đề tài này, em sử dụng mô hình học sâu YOLO. Với khả năng trích xuất đặc trưng đa tỷ lệ và xử lý ảnh trực tiếp qua một lần quét, mô hình có thể nhận diện và phân loại chính xác các vật thể khác nhau, đồng thời đảm bảo được tốc độ xử lý thời gian thực.

1.4.2 Phương pháp theo dõi đa đối tượng

Trong các bài toán phát hiện vật thể tĩnh truyền thống, mô hình chỉ phân tích từng khung hình (frame) một cách độc lập và ánh xạ ma trận pixel thành các tọa độ Bounding Box rời rạc. Tuy nhiên, luồng giao thông thực tế là một dòng chảy thời gian liên tục. Nếu chỉ sử dụng phát hiện độc lập, cùng một chiếc xe trong hai khung hình liên tiếp sẽ bị hệ thống đếm sai thành hai chiếc xe khác nhau.

Do đó, đề tài này tiếp cận bài toán Theo dõi Đa đối tượng (Multi-Object Tracking - MOT). Về mặt logic, bên cạnh việc học một hàm quyết định để phân loại lớp ý và tọa độ tại thời điểm t , hệ thống phải giải quyết bài toán liên kết dữ liệu (Data Association). Khi một phương tiện xuất hiện, hệ thống cấp cho nó một định danh duy nhất (ID). Thông qua thuật toán theo dõi ByteTrack, hệ thống sẽ duy trì quỹ đạo của xe qua từng khung hình, bất kể việc phương tiện bị thay đổi hình dạng hay bị che khuất tạm thời. Nhờ đó mà hệ thống có thể đếm chính xác số lượng phương tiện đang lưu thông.

1.4.3 Phương pháp đếm số lượng phương tiện giao thông

Sau khi các phương tiện được nhận diện và duy trì mã định danh (ID) xuyên suốt các khung hình trong video, bài toán cuối cùng là đếm lưu lượng giao thông.

Trong đề tài này, phương pháp đếm dựa trên Vùng quan tâm (Region of Interest) được sử dụng cho từng làn đường riêng biệt. Hệ thống sẽ liên tục kiểm tra sự thay đổi của tọa độ trọng tâm của phương tiện theo thời gian để ghi nhận trạng thái phương tiện đang tiến vào hay rời khỏi vùng đếm. Cùng với đó, trong các điều kiện thời tiết khắc nghiệt, hình ảnh phương tiện thường bị nhiễu hoặc bị che khuất bởi mưa, sương mù,..., khiến hệ thống theo dõi tạo ra nhiều mã ID ảo cho cùng một phương tiện, gây ra hiện tượng phương tiện đó bị đếm lặp lại nhiều lần. Để khắc phục, đề tài sử dụng bộ lọc nhiễu với cơ chế loại bỏ các ID ảo khỏi hệ thống đếm, đảm bảo sự chính xác của quá trình đếm lưu lượng giao thông.

Chương 2 CƠ SỞ LÝ THUYẾT

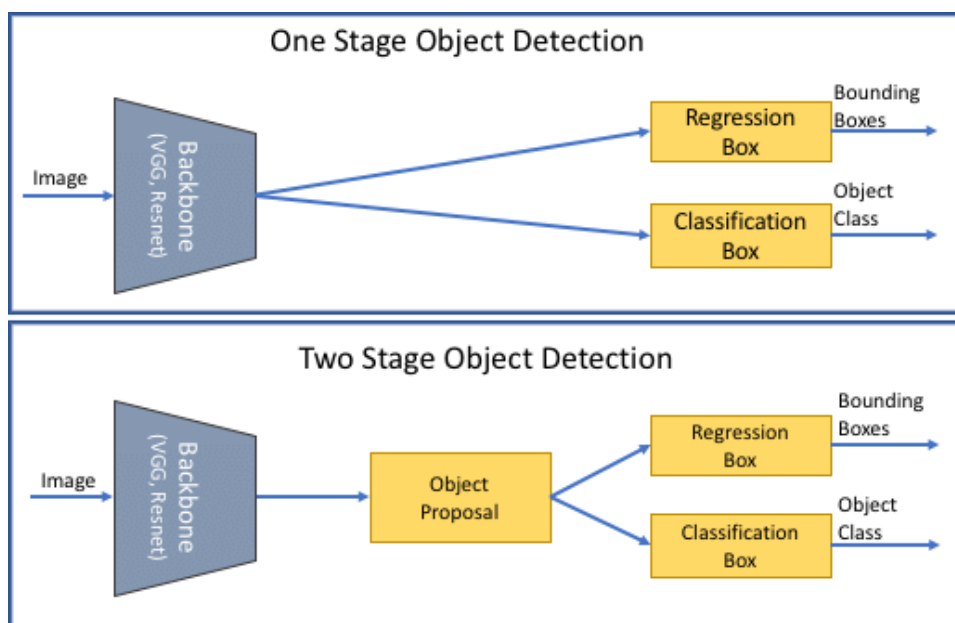
2.1 Bài toán nhận diện vật thể

Nhận diện vật thể (Object Detection) là một trong những bài toán thuộc lĩnh vực thị giác máy tính. Mục tiêu của bài toán là phát hiện sự hiện diện của vật thể, xác định vị trí của chúng trong ảnh, video và phân loại vào các lớp khác nhau.

Đầu ra của một mô hình nhận diện vật thể tiêu chuẩn bao gồm ba thông tin chính:

- **Hộp giới hạn (Bounding Box):** Thường được biểu diễn dưới dạng tọa độ (x, y, w, h) , trong đó (x, y) là tọa độ tâm của hộp, w là chiều rộng và h là chiều cao.
- **Nhãn lớp (Class Label):** Phân loại vật thể thuộc nhóm nào (Ví dụ: Car, Truck, Pedestrian).
- **Độ tự tin (Confidence Score):** Giá trị xác suất biểu thị mức độ chắc chắn của mô hình về dự đoán của mình.

Trong những năm gần đây, sự phát triển của Mạng nơ-ron tích chập (CNN) đã chia bài toán này thành hai mô hình: **Mô hình hai giai đoạn (Two-stage Detectors)** điển hình là Faster R-CNN có độ chính xác cao nhưng xử lý chậm, và **mô hình một giai đoạn (One-stage Detectors)** điển hình là YOLO dự đoán trực tiếp tọa độ và nhãn lớp trong một lần quét, đáp ứng yêu cầu xử lý thời gian thực của các hệ thống camera giao thông [1].



Hình 2.1: Mô hình một giai đoạn và mô hình hai giai đoạn.

2.1.1 Mô hình hai giai đoạn

Quá trình nhận diện được chia làm hai bước tuần tự tách biệt:

- **Giai đoạn 1 - Trích xuất vùng đề xuất (Region Proposals):** Thay vì quét toàn bộ ảnh, mô hình tạo ra một tập hợp các đề xuất về các khu vực có khả năng chứa đối tượng.
- **Giai đoạn 2 - Phân loại và Tinh chỉnh (Classification & Regression):** Các vùng đề xuất từ giai đoạn 1 sẽ được cắt ra và đưa qua một mạng nơ-ron khác để thực hiện hai nhiệm vụ song song: (1) Phân loại xem vật thể trong vùng đó thuộc lớp nào và (2) hồi quy tọa độ hộp giới hạn (Bounding Box Regression) để khớp chính xác với vật thể.

Việc chia nhỏ bài toán giúp mô hình hai giai đoạn đạt được độ chính xác (Accuracy) cao, khả năng nhận diện vật thể nhỏ tốt. Tuy nhiên, do phải thực hiện hai bước tính toán, tốc độ xử lý chậm hơn mô hình một giai đoạn [1].

2.1.2 Mô hình một giai đoạn

Mô hình một giai đoạn loại bỏ việc tìm kiếm vùng đề xuất, chuyển bài toán nhận diện vật thể thành một bài toán hồi quy duy nhất (Single Regression Problem).

Bức ảnh đầu vào chỉ đi qua mạng nơ-ron tích chập đúng một lần duy nhất (Single pass). Thuật toán sẽ chia bức ảnh thành một lưới (Grid). Tại mỗi ô lưới, mạng nơ-ron sẽ dự đoán trực tiếp tọa độ hộp giới hạn (Bounding Box) và xác suất phân lớp (Class probabilities) cùng một lúc cho toàn bộ bức ảnh.

Bằng cách loại bỏ hoàn toàn bước trích xuất vùng đề xuất, kiến trúc một giai đoạn tối ưu hóa luồng dữ liệu, đạt được tốc độ thời gian thực (Real-time speed). Dù trước đây kiến trúc này có độ chính xác thấp hơn, nhưng với các tiến bộ mới của các phiên bản sau, nhược điểm này đã được khắc phục [1].

2.1.3 Giới thiệu về mô hình YOLO

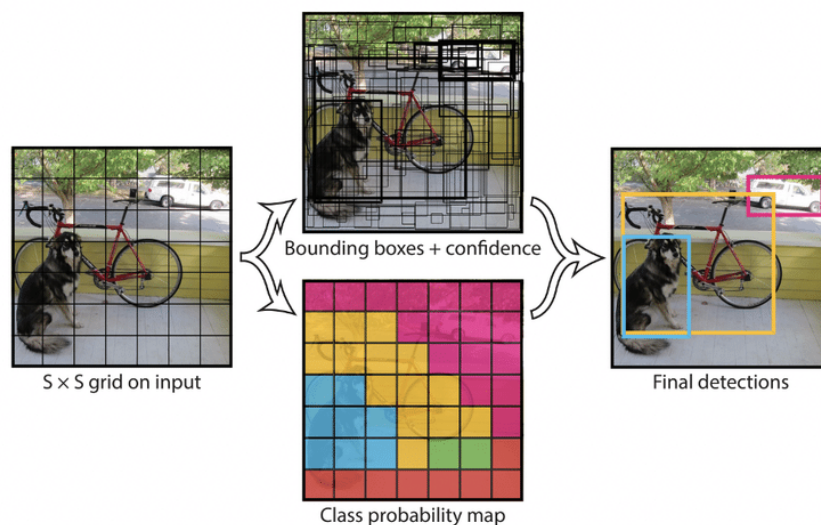
YOLO (You Only Look Once) là họ mô hình nhận diện vật thể một giai đoạn tiên tiến nhất hiện nay, nổi bật với tốc độ nhanh chóng và khả năng nhận diện đối tượng trong thời gian thực, được sử dụng trong nhiều ứng dụng thực tế như giám sát an ninh, lái xe tự động, và nhận diện biển số xe [2], [3].

2.1.3.1 Nguyên lý cơ bản của YOLO

Thay vì quét ảnh nhiều lần, thuật toán YOLO chia bức ảnh đầu vào thành một lưới (Grid) kích thước $S \times S$, trong đó mỗi ô lưới chịu trách nhiệm phát hiện các đối tượng có tọa độ tâm rơi vào bên trong ô đó. Cụ thể, quá trình này diễn ra như sau:

- **Chia hình ảnh thành lưới (Grid Division):** Hình ảnh đầu vào được chia thành một lưới có kích thước $S \times S$. Mỗi ô lưới sẽ đảm nhận việc dự đoán B hộp giới hạn (Bounding Boxes) và xác suất của các lớp đối tượng.

- **Dự đoán hộp giới hạn (Bounding Box Prediction):** Mỗi hộp giới hạn được biểu diễn bởi 5 thông số cốt lõi: tọa độ tâm (x,y) , chiều rộng w , chiều cao h của hộp, và giá trị độ tin cậy (Confidence score) thể hiện mức độ chắc chắn của mô hình về việc có đối tượng tồn tại trong hộp đó.
- **Bản đồ xác suất lớp (Class Probability Map):** Song song với việc dự đoán hộp giới hạn, mỗi ô lưới cũng tính toán xác suất có điều kiện của từng lớp đối tượng nếu giả định rằng có một đối tượng đang hiện diện trong ô đó.
- **Kết hợp dự đoán (Prediction Fusion):** Thuật toán YOLO sẽ nhân giá trị độ tin cậy của hộp giới hạn với bản đồ xác suất lớp. Kết quả cuối cùng tạo ra một tập hợp các Bounding Box đi kèm nhãn và điểm số, từ đó quyết định vị trí, kích thước và phân loại của đối tượng trên toàn bộ bức ảnh.



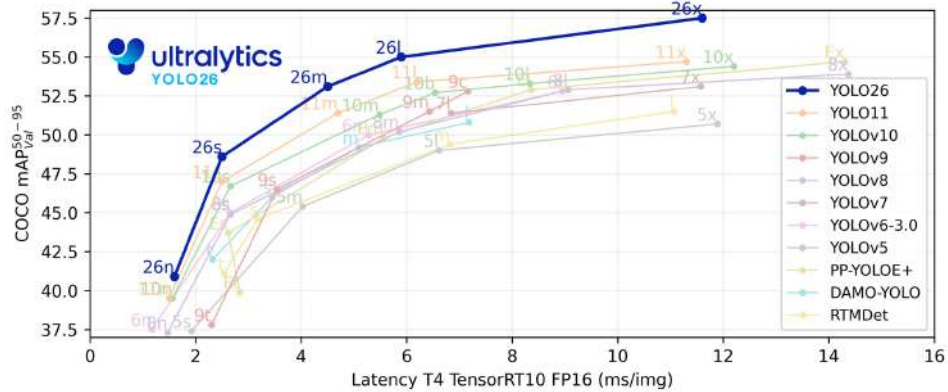
Hình 2.2: Nguyên lý cơ bản của YOLO.

2.1.3.2 Các phiên bản YOLO

Tính đến thời điểm hiện tại, họ mô hình YOLO đã trải qua nhiều sự thay đổi về kiến trúc nhằm tối ưu hóa giữa tốc độ và độ chính xác: [3]

- **YOLOv1 (2015):** Phiên bản đầu tiên đặt nền móng cho kiến trúc nhận diện vật thể thời gian thực bằng cách chuyển bài toán nhận diện sang dạng hồi quy đơn lẻ.
- **YOLOv2 (2016):** Cải tiến lớn nhờ tích hợp cơ chế Anchor Boxes, chuẩn hóa hàng loạt (Batch Normalization) và khả năng nhận diện hơn 9000 loại đối tượng.
- **YOLOv3 (2018):** Sử dụng mạng Darknet-53, bổ sung dự đoán đa quy mô (Multi-scale predictions) giúp tối ưu đáng kể khả năng nhận diện vật thể nhỏ.
- **YOLOv4 (2020):** Tối ưu hóa các kỹ thuật tăng cường dữ liệu và kiến trúc mạng (Cơ chế Bag-of-Freebies & Bag-of-Specials).
- **YOLOv5 (2020):** Phát triển trên framework PyTorch bởi Ultralytics, đơn giản hóa quy trình huấn luyện và triển khai mô hình.
- **YOLOv6 (2022):** Do tập đoàn Meituan (Trung Quốc) phát triển, định hướng tối ưu phần cứng cho các thiết bị công nghiệp và robot tự hành.
- **YOLOv7 (2022):** Tập trung tối ưu hóa kiến trúc mạng để đạt tốc độ và độ chính xác vượt trội tại thời điểm ra mắt.
- **YOLOv8 (2023):** Chuyển đổi YOLO thành một framework đa tác vụ gồm: Nhận diện (Detection), phân đoạn (Instance segmentation), phân loại (Classification) và ước tính tư thế (Pose estimation).
- **YOLOv9 (2024):** Giới thiệu cơ chế Thông tin gradient lập trình được (PGI) để khắc phục tình trạng mất mát thông tin trong mạng sâu.
- **YOLOv10 (2024):** Loại bỏ hoàn toàn thuật toán loại bỏ vật thể trùng lặp NMS (Non-Maximum Suppression), giúp giảm đáng kể độ trễ khi suy luận.
- **YOLO11 (2024):** Tối ưu hóa sâu kiến trúc mạng, giảm lượng tham số nhưng tăng độ chính xác trên nhiều tác vụ thị giác.

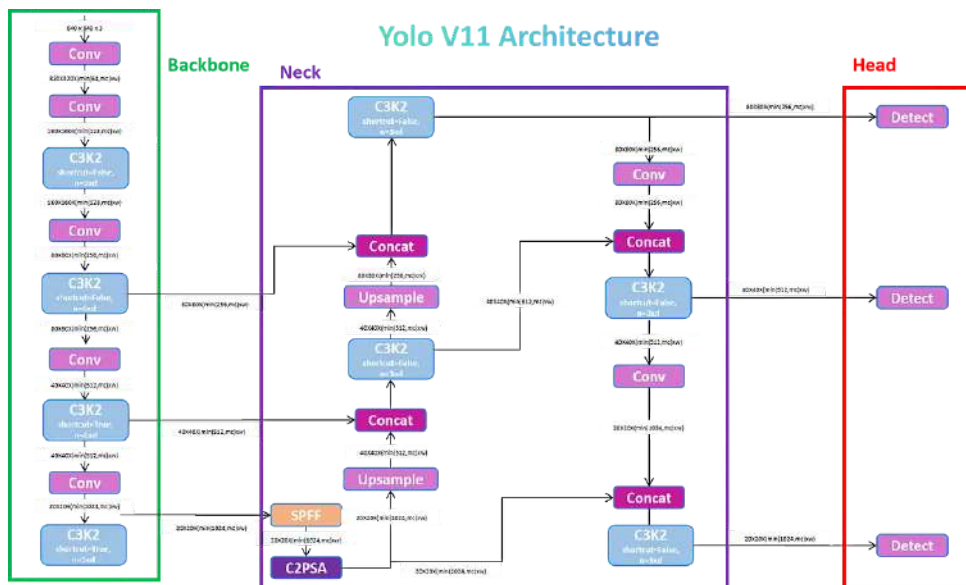
- **YOLO26 (2026):** Phiên bản mới nhất từ Ultralytics, loại bỏ lớp DFL, thiết kế NMS-Free và chuyên biệt hóa cho việc triển khai trên các thiết bị biên (Edge devices) có phần cứng hạn chế.



Hình 2.3: Biểu đồ so sánh độ chính xác (COCO mAP) và độ trễ (Latency) giữa các thể hệ YOLO.

2.1.3.3 Kiến trúc YOLO 11

Phiên bản YOLO11 được phát triển bởi Ultralytics, tiếp tục kế thừa và tối ưu hóa kiến trúc cốt lõi của các phiên bản YOLOv8 và YOLOv10 để nâng cao hiệu năng trích xuất đặc trưng, đặc biệt là khả năng phát hiện các vật thể nhỏ trong điều kiện nhiễu loạn [4].



Hình 2.4: Kiến trúc YOLO11.

Cấu trúc lõi của mạng YOLO11 bao gồm ba phần chính:

- **Backbone (Xương sống):** Đóng vai trò trích xuất các đặc trưng từ ảnh đầu vào. YOLO11 sử dụng các khối tích chập tối ưu hóa để trích xuất từ các đặc trưng bậc thấp (cạnh, màu sắc) đến các đặc trưng bậc cao (hình dáng, kết cấu của vật thể), tạo ra các bản đồ đặc trưng ở nhiều độ phân giải khác nhau:
 - **Lớp tích chập (Convolutional Layers):** YOLO11 duy trì cấu trúc tương tự như các phiên bản trước, sử dụng các lớp tích chập ban đầu để giảm mẫu ảnh. Những lớp này tạo nền tảng cho quá trình trích xuất đặc trưng, giảm dần kích thước không gian trong khi tăng số lượng kênh.
 - **Khối C3k2:** Một cải tiến đáng kể trong YOLO11 là sự ra đời của khối C3k2, thay thế khối C2f được sử dụng trong các phiên bản trước. Khối C3k2 là một sự cải tiến về mặt tính toán của CSP (Cross Stage Partial) Bottleneck. Thay vì sử dụng một phép tích chập lớn như trong YOLOv8, C3k2 áp dụng hai phép tích chập 3x3 nhỏ hơn để tối ưu luồng thông tin và giảm chi phí tính toán, giúp tăng tốc độ xử lý trong khi vẫn duy trì khả năng nắm bắt các đặc trưng quan trọng.
- **Neck (Cổ mạng):** Là phần trung gian giữa Backbone và Head, đóng vai trò thu thập và kết hợp các bản đồ đặc trưng (Feature Maps) từ nhiều tầng khác nhau của Backbone và truyền chúng đến Head để chuẩn bị cho việc dự đoán. Cơ chế này giúp mô hình hiệu quả hơn trong việc thu thập thông tin đa tỷ lệ.
 - **Khối C3k2:** Khối C3k2 được thiết kế nhanh hơn và hiệu quả hơn, góp phần nâng cao hiệu suất tổng thể của quá trình tổng hợp đặc trưng. Sau khi thực hiện tăng mẫu và kết hợp, phần Neck trong YOLO11 tích hợp khối cải tiến này, dẫn đến tốc độ và hiệu suất tốt hơn.

- **Khối SPPF (Spatial Pyramid Pooling Fast):** Module này cho phép mạng tổng hợp thông tin bối cảnh từ các vùng có kích thước khác nhau, đặc biệt quan trọng để cải thiện khả năng phát hiện các đối tượng với nhiều kích cỡ khác nhau. YOLO11 vẫn giữ lại khối SPPF từ các phiên bản trước, nhưng bổ sung một khối mới là C2PSA ngay sau đó.
- **Khối C2PSA (Cross Stage Partial Spatial Attention):** là một cải tiến đáng kể giúp tăng cường khả năng chú ý không gian trên các bản đồ đặc trưng. Cơ chế này giúp mô hình tập trung hiệu quả hơn vào các vùng quan trọng trong hình ảnh, giúp cải thiện độ chính xác trong việc phát hiện, đặc biệt đối với các đối tượng nhỏ hoặc bị che khuất một phần. Việc tích hợp C2PSA làm cho YOLO11 khác biệt so với phiên bản tiền nhiệm, YOLOv8, vốn không có cơ chế chú ý cụ thể này.
- **Head (Đầu ra):** Sử dụng các đặc trưng từ Neck nhằm thực hiện các dự đoán cuối cùng để xuất ra tọa độ Bounding Box và xác suất phân lớp mà không cần phụ thuộc vào các hộp neo (Anchor-free).

2.2 Bài toán theo dõi đa đối tượng

Nhận diện chính xác chỉ là bước đầu tiên. Để đếm được lưu lượng giao thông, hệ thống cần giải quyết bài toán Theo dõi đa đối tượng (MOT), đảm bảo Bounding Box của các phương tiện được duy trì nhất quán xuyên suốt các khung hình liên tiếp trong video.

Phương pháp tiếp cận phổ biến hiện nay là Tracking-by-detection. Hệ thống sử dụng các thuật toán theo dõi tiên tiến (như ByteTrack hoặc BoT-SORT) tích hợp sẵn Bộ lọc Kalman (Kalman Filter) để dự đoán quỹ đạo chuyển động học của phương tiện ở khung hình tiếp theo. Sau đó, thuật toán Hungary (Hungarian Algorithm) được sử dụng để ghép Bounding Box của khung hình hiện tại với khung hình trước đó.

Đặc biệt, ByteTrack mang lại ưu thế lớn trong thời tiết xấu nhờ cơ chế duy trì các Bounding Box có độ tự tin thấp. Thay vì loại bỏ những dự đoán không chắc chắn khi một phương tiện di chuyển trong vùng bị ảnh hưởng bởi điều kiện thời tiết khắc nghiệt, thuật toán vẫn giữ lại và liên kết chúng với quỹ đạo cũ, giúp ID của phương tiện không bị đứt đoạn.

2.2.1 Bộ lọc Kalman

Bộ lọc Kalman (Kalman Filter) [5] là một thuật toán đệ quy toán học dùng để ước lượng và dự đoán trạng thái tương lai của một hệ thống động (như tọa độ và vận tốc của xe) trong môi trường có nhiễu nhiễu.

Trạng thái của mỗi đối tượng thường được biểu diễn bằng vector:

$$x = [u, v, s, r, \dot{u}, \dot{v}, \dot{s}]$$

Trong đó (u, v) là tọa độ tâm, s là diện tích, r là tỷ lệ khung hình, và các biến $\dot{u}, \dot{v}, \dot{s}$ là các thành phần vận tốc.

Tại mỗi khung hình (frame), Bộ lọc Kalman thực hiện hai bước:

- **Dự đoán (Predict):** Dựa vào trạng thái và vận tốc ở frame $t - 1$, thuật toán sẽ dự đoán vị trí tiếp theo ở frame t .
- **Cập nhật (Update):** Khi mạng YOLO trả về kết quả phát hiện thực tế ở frame t , bộ lọc Kalman sẽ kết hợp giữa "dự đoán" và "thực tế" để tự điều chỉnh lại tọa độ cho chính xác, chuẩn bị cho frame tiếp theo.

2.2.2 Thuật toán Hungarian

Đây là một thuật toán tối ưu hóa tổ hợp để giải quyết bài toán gán tối ưu (Assignment Problem) trong thời gian đa thức. Giả sử tại frame t , thuật toán Kalman dự đoán được 5

quỹ đạo cũ (Tracks), nhưng mạng YOLO lại phát hiện ra 6 Bounding Box mới (Detections). Để biết Box nào thuộc về Track nào, thuật toán sẽ lập một "Ma trận chi phí" (Cost Matrix) giữa tất cả các Tracks và Detections. Chi phí này thường được tính dựa trên mức độ giao thoa hình học IoU (Intersection over Union). Dựa trên ma trận này, Thuật toán Hungarian sẽ tìm ra phương pháp đối khớp tối ưu giữa Track và Detection sao cho tổng chi phí của toàn bộ hệ thống là nhỏ nhất (tức là tổng IoU lớn nhất) [5].

2.2.3 Nguyên lý hoạt động của ByteTrack

Các thuật toán theo dõi truyền thống (như SORT hay DeepSORT) thường chỉ lấy các Bounding Box có điểm tự tin cao để đưa vào thuật toán Hungarian và loại bỏ hoàn toàn các Bounding Box có điểm thấp. Điều này cực kỳ rủi ro trong thời tiết xấu hoặc khi phương tiện bị che khuất (Occlusion), vì khi đó điểm tự tin của YOLO bị tụt xuống rất thấp, làm đứt gãy ID theo dõi.

Điểm khác biệt chính của ByteTrack so với các phương pháp trên là cơ chế sử dụng tất cả các detection, kể cả những detection có confidence thấp [5].

Nguyên lý hoạt động của ByteTrack:

- **Giai đoạn 1:** Sử dụng những Detection có độ tự tin cao để ghép cặp (bằng Hungarian) với những Track cũ (thông qua IoU matching và bộ lọc Kalman).
- **Giai đoạn 2:** Những Detection có độ tự tin thấp sẽ được ghép cặp với những Track cũ chưa được ghép cặp ở giai đoạn 1. Điều này giúp duy trì quỹ đạo trong trường hợp đối tượng bị che khuất hoặc mô hình nhận diện kém.
- **Giai đoạn 3:** Sau Giai đoạn 2, đối với các quỹ đạo (Tracks) vẫn không thể đối khớp thành công, ByteTrack không lập tức loại bỏ mà sẽ chuyển chúng sang trạng thái "mất dấu" (Lost Tracks) nhằm duy trì khả năng khôi phục trong trường hợp phương tiện bị che khuất tạm thời. Đồng thời, đối với các kết quả nhận diện (Detections) có

độ tin cậy cao nhưng không tìm được Track cũ tương ứng để ghép cặp, hệ thống sẽ tự động khởi tạo các quỹ đạo theo dõi mới. Kết quả cuối cùng tại mỗi khung hình đầu ra là một tập hợp các đối tượng được gán mã định danh (ID) liên tục, đi kèm với thông tin về tọa độ hộp giới hạn và nhãn phân lớp.

2.3 Các chỉ số đánh giá mô hình

Để định lượng khách quan hiệu năng của mô hình nhận diện vật thể và hệ thống đếm phương tiện giao thông, đề tài sử dụng các chỉ số toán học tiêu chuẩn sau:

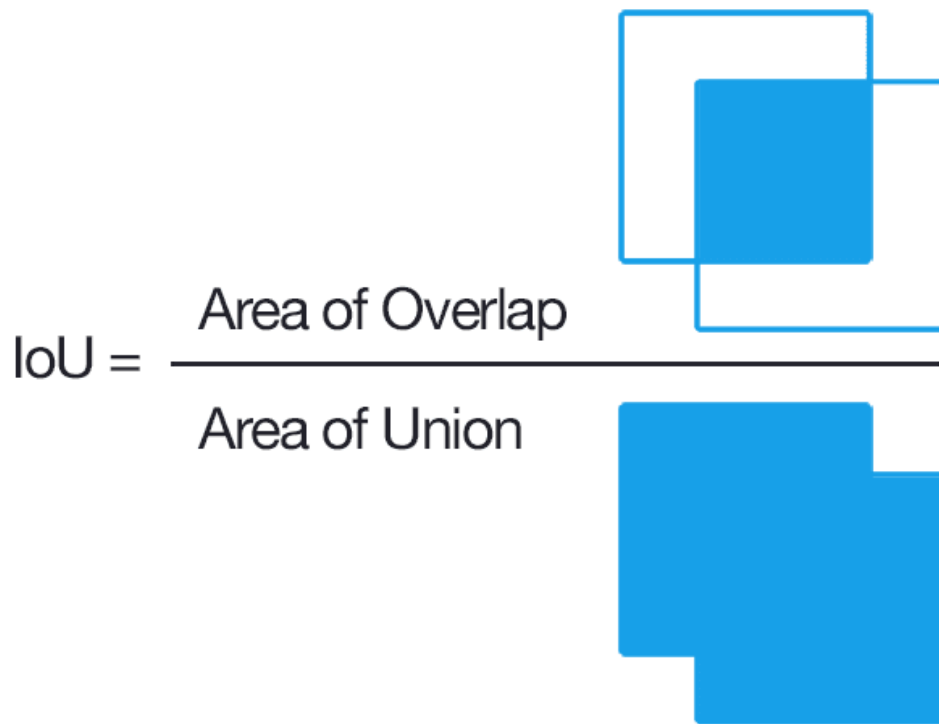
2.3.1 *Intersection over Union (IoU)*

IoU là thước đo định lượng mức độ chồng lấp (overlap) giữa bounding box dự đoán và bounding box thực tế (ground truth).

$$IoU = \frac{\text{Area}(B_p \cap B_{gt})}{\text{Area}(B_p \cup B_{gt})} \quad (2.1)$$

Trong đó:

- B_p : Diện tích của Bounding Box dự đoán.
- B_{gt} : Diện tích của Bounding Box thực tế.



Hình 2.5: Intersection over Union.

Việc xác định IoU giúp tính toán khả năng phát hiện chính xác vật thể. Nếu giá trị IoU vượt qua một ngưỡng định trước (Ví dụ: 0.5), dự đoán được coi là chính xác (True Positive).

2.3.2 *Precision (Độ chính xác) và Recall (Độ bao phủ)*

Precision: Tỷ lệ các phát hiện đúng trong tổng số phát hiện của mô hình. Precision cao nghĩa là ít phát hiện sai (false positive).

$$Precision = \frac{TP}{TP + FP} \quad (2.2)$$

Recall: Tỷ lệ đối tượng thực sự được phát hiện trong tổng số đối tượng thực tế. Recall cao nghĩa là ít bỏ sót (false negative)

$$Recall = \frac{TP}{TP + FN} \quad (2.3)$$

Trong đó:

- TP (True Positive): Số trường hợp được mô hình nhận diện là đúng và thực tế là đúng.
- FP (False Positive): Số trường hợp được mô hình nhận diện là đúng nhưng thực tế là sai.
- FN (False Negative): Số trường hợp được mô hình nhận diện là sai nhưng thực tế là đúng.

2.3.3 *F1-Score*

Vì Precision và Recall thường có xu hướng đánh đổi lẫn nhau, F1-Score được sử dụng làm trung bình điều hòa của cả hai chỉ số, giúp đánh giá tổng thể độ cân bằng của hệ thống.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (2.4)$$

2.3.4 *Average Precision (AP) và Mean Average Precision (mAP)*

Chỉ số Average Precision (AP) của một lớp được tính bằng diện tích phần dưới đường cong Precision-Recall:

$$AP = \int_0^1 p(r) dr \quad (2.5)$$

mAP là giá trị trung bình cộng của *AP* trên tất cả các lớp vật thể (từ lớp 1 đến lớp N)

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i \quad (2.6)$$

Trong đó:

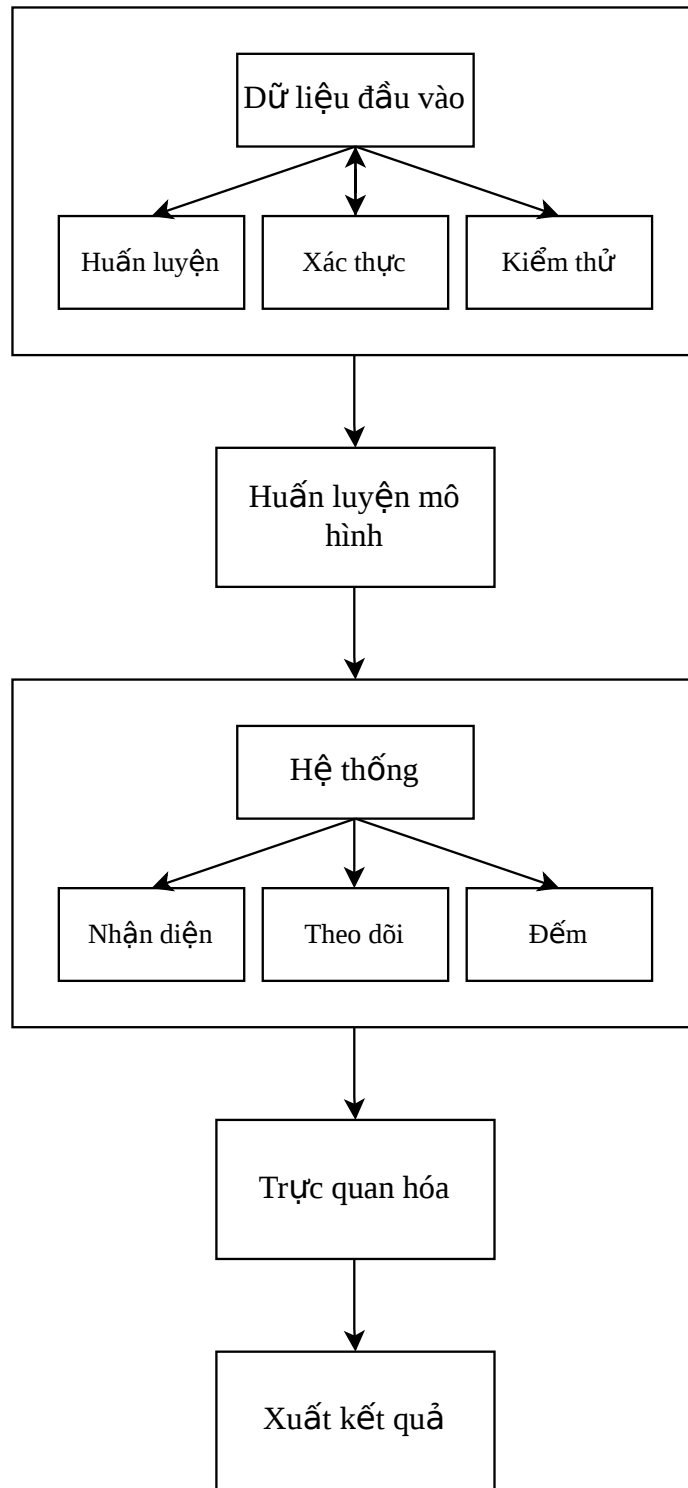
- $mAP@50$ là mAP tính tại ngưỡng $IoU = 0.5$
- $mAP@50 - 95$ là trung bình mAP tính tại các ngưỡng IoU từ 0.5 đến 0.95 (bước nhảy 0.05)

Chương 3 THỰC NGHIỆM

3.1 Kiến trúc tổng quan của hệ thống

Để giải quyết bài toán nhận diện và đếm phương tiện trong điều kiện khắc nghiệt, đề tài xây dựng và triển khai hệ thống dựa trên một quy trình tuần tự như sau:

1. **Chuẩn bị dữ liệu:** Hệ thống tiếp nhận và phân chia dữ liệu đầu vào thành ba tập riêng biệt: tập Huấn luyện (Training set), tập Xác thực (Validation set) và tập Kiểm thử (Test set).
2. **Huấn luyện mô hình:** Dữ liệu được nạp vào mạng nơ-ron YOLO11m để trích xuất các đặc trưng của đối tượng.
3. **Tích hợp vào hệ thống:** Mô hình sau huấn luyện được tích hợp vào luồng xử lý, kết hợp với thuật toán ByteTrack để nhận diện, theo dõi và duy trì mã định danh liên tục cho từng phương tiện.
4. **Khởi tạo các vùng quan tâm:** Thiết lập các tọa độ của các đa giác trên khung hình video. Các vùng này đóng vai trò là ranh giới để kích hoạt sự kiện đếm.
5. **Thiết kế logic đếm xe:** Khi một phương tiện nằm trong vùng quan tâm, hệ thống sẽ ghi nhận trạng thái luồng (Đi vào hay đi ra) và tự động tăng biến đếm tương ứng.
6. **Trực quan hóa:** Hiển thị hộp giới hạn (Bounding box), nhãn phân loại (Class label), mã định danh (Track ID) và thống kê lưu lượng phương tiện đang lưu thông lên video.
7. **Xuất kết quả:** Video sau quá trình xử lý (đã bao gồm các hiển thị trực quan) được xuất ra và lưu trữ dưới định dạng tiêu chuẩn mp4.



Hình 3.1: Sơ đồ kiến trúc tổng quan của hệ thống

3.2 Huấn luyện mô hình

3.2.1 Chuẩn bị dữ liệu

Đề tài sử dụng bộ dữ liệu TSBOW (Traffic Surveillance Benchmark for Occluded Vehicles Under Various Weather Conditions) [6] do nhóm nghiên cứu Automation Lab thuộc khoa Kỹ thuật Điện và Máy tính (Department of Electrical and Computer Engineering), Đại học Sungkyunkwan, tỉnh Suwon, Hàn Quốc phát triển và công bố tại hội nghị AAAI-26. Khác với các bộ dữ liệu giao thông thông thường chủ yếu được quay ở góc thấp, TSBOW được thiết kế chuyên biệt cho góc nhìn camera giám sát trên cao. Bộ dữ liệu này tập trung giải quyết những thách thức đến từ sự che khuất phương tiện với mật độ dày đặc và sự suy giảm chất lượng hình ảnh do các hiện tượng thời tiết khắc nghiệt như mưa, tuyết, sương mù và thiếu sáng.

Để phục vụ cho quá trình huấn luyện, toàn bộ dữ liệu huấn luyện và kiểm thử của đề tài được truy xuất và tải về trực tiếp từ kho lưu trữ mã nguồn mở Hugging Face do chính nhóm tác giả cung cấp.



Hình 3.2: Bộ dữ liệu TSBOW.

3.2.1.1 Thống kê dữ liệu

TSBOW là một bộ dữ liệu quy mô lớn (Large-scale benchmark), vượt trội về số lượng và chất lượng so với các tập dữ liệu về giao thông. Các thống kê nổi bật bao gồm: [6]

- **Dung lượng video:** Bao gồm 198 video camera giám sát với tổng thời lượng lên tới 32.36 giờ quay thực tế.
- **Chất lượng hiển thị:** Toàn bộ dữ liệu được chuẩn hóa ở độ phân giải tiêu chuẩn cao 1280x720 pixel.
- **Số lượng khung hình và gán nhãn:** Tập dữ liệu sở hữu tổng cộng 3.2 triệu khung hình. Trong đó, để đảm bảo độ chính xác cho việc đánh giá, có hơn 48.000 khung hình được gán nhãn thủ công (Manually Labeled). Quá trình này đã tạo ra 1.1 triệu hộp giới hạn (Bounding Boxes). Ngoài ra, có tới 71.1 triệu hộp giới hạn được gán nhãn bán tự động để phục vụ cho các mô hình học sâu lớn.

198 Processed Videos	32 h Duration	3.2 M Total Frames
71.1 M Semi-Annotated Instances	48 K Manual-Annotated Frames	1.1 M Manual-Annotated Instances

Hình 3.3: Thống kê dữ liệu TSBOW.

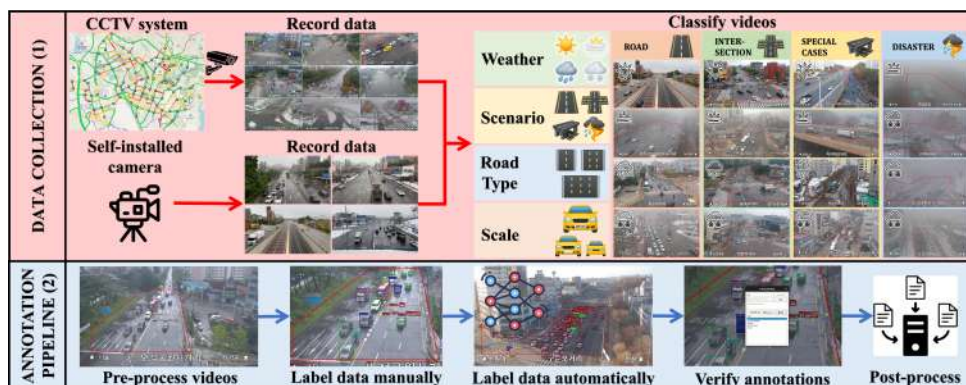
3.2.1.2 Phân bố nhãn và thuộc tính dữ liệu

Dữ liệu được thu thập trên 71 tuyến đường và ngã tư khác nhau, mang lại tính đa dạng cao về bối cảnh [6].

Về phân bố nhãn (Classes): Các phương tiện được phân loại thành 8 lớp đối tượng: **Car (Ô tô con)**, **Bus (Xe buýt)**, **Truck (Xe tải lớn)**, **Small truck (Xe tải nhỏ)**, **Pedestrian (Người đi bộ)**, **Micromobility (Phương tiện cỡ nhỏ như xe máy/xe đạp)**, **Unidentified (Không xác định)**, và **Others (Khác)**. Đặc điểm của phân bố này là sự chênh lệch kích thước vô cùng lớn giữa các phương tiện, đòi hỏi hệ thống phải có khả năng trích xuất đặc trưng đa tỷ lệ [6].

Về thuộc tính dữ liệu (Attributes): Môi trường của tập dữ liệu được gán nhãn theo 4 điều kiện thời tiết khác nhau: [6]

- **Thời tiết (Weather):** Normal (Nắng/Nhiều mây), Haze (Sương mù), Rain (Mưa), và Snow (Tuyết rơi).
- **Bối cảnh (Scenario):** Road (Đoạn đường thẳng), Intersection (Ngã tư giao cắt), Special cases (Trường hợp đặc biệt), và Disaster (Thảm họa thời tiết).
- **Đặc điểm đường (Road Type):** Urban (Đô thị), Standard (Tiêu chuẩn), và Boulevard (Đại lộ).
- **Góc nhìn (Scale):** Fine (Gần/Chi tiết), Medium (Trung bình), và Coarse (Xa/Bao quát)



Hình 3.4: Thuộc tính dữ liệu.

3.2.2 *Môi trường và công cụ thực nghiệm*

3.2.2.1 Cấu hình phần cứng

Quá trình huấn luyện và triển khai được thực hiện trên nền tảng điện toán đám mây Google Colab với bộ xử lý đồ họa (GPU) NVIDIA Tesla T4 với 15GB VRAM và bộ xử lý trung tâm (CPU) Intel Xeon 2.00GHz với 12.7GB RAM hệ thống.

3.2.2.2 Môi trường phần mềm

Ngôn ngữ lập trình: Python 3 (phiên bản tiêu chuẩn của môi trường Colab).

Nền tảng tính toán: Khung làm việc học sâu (Deep Learning Framework) PyTorch, được tích hợp sẵn bộ công cụ NVIDIA CUDA Toolkit để kích hoạt khả năng tăng tốc phần cứng trên GPU.

Thư viện triển khai AI: Ultralytics – một hệ sinh thái mã nguồn mở, được sử dụng để khởi tạo cấu trúc mạng YOLO, tinh chỉnh siêu tham số (Hyperparameters) và tích hợp thuật toán theo dõi quỹ đạo ByteTrack.

Thư viện xử lý thị giác: OpenCV (Open Source Computer Vision Library): Đảm nhiệm việc đọc/ghi luồng video, xử lý từng khung hình, và nội suy đồ họa.

Thư viện tính toán số học: NumPy: Được sử dụng để xử lý các phép toán trên ma trận và quản lý mảng dữ liệu.

3.2.3 *Huấn luyện*

Quá trình huấn luyện đòi hỏi sự cân bằng giữa kích thước đầu vào, độ phức tạp của mô hình và giới hạn phần cứng. Đề tài thiết lập cấu hình huấn luyện tối ưu như sau:

- **Phân chia tập dữ liệu:** Mặc dù bộ dữ liệu TSBOW gốc sở hữu hơn 3.2 triệu khung hình video, tuy nhiên để tối ưu hóa thời gian huấn luyện và tránh hiện tượng mạng

YOLO bị học vẹt (Overfitting) do các khung hình liên kế có sự tương đồng với nhau, đề tài sử dụng phiên bản tập dữ liệu đã được trích xuất mẫu (Sampling) với tổng số 25.114 hình ảnh. Toàn bộ dữ liệu đã được nhóm tác giả cấu trúc sẵn thành 3 tập độc lập, với tỷ lệ phân bổ như sau:

Bảng 3.1: Cấu hình phân chia tập dữ liệu thực nghiệm

Phân chia dữ liệu	Số lượng	Tỷ lệ	Mục đích sử dụng
Tập huấn luyện	10.917	43,5%	Trích xuất đặc trưng hình học và cập nhật ma trận trọng số.
Tập xác thực	7.583	30,2%	Đánh giá chéo sau mỗi vòng lặp, tính toán sai số để lưu lại mô hình có độ chính xác cao nhất.
Tập kiểm thử	6.614	26,3%	Kiểm tra và đánh giá độ chính xác khách quan của mô hình.

- **Cấu hình siêu tham số (Hyperparameters):** Quá trình huấn luyện được tinh chỉnh qua các tham số sau:
 - **Mô hình huấn luyện:** Lựa chọn YOLO11 phiên bản Medium nhằm tối ưu giữa tốc độ và độ chính xác trong quá trình huấn luyện.
 - **Kích thước ảnh đầu vào:** Sử dụng ảnh có độ phân giải đầu vào là 1280x1280. Thiết lập này nhằm giữ lại các đặc trưng điểm ảnh của các vật thể nhỏ trong môi trường bị ảnh hưởng bởi thời tiết khắc nghiệt.
 - **Chu kỳ huấn luyện:** Quá trình huấn luyện được thiết lập tối đa 50 vòng. Bên cạnh đó còn có cơ chế dừng sớm (Early Stopping), mô hình sẽ dừng huấn luyện nếu chỉ số *mAP* trên tập Xác thực (Validation) không cải thiện sau 10 vòng liên tiếp. Điều này giúp ngăn chặn hiện tượng quá khớp (Overfitting).
 - **Kích thước mẫu huấn luyện:** *Batch_size* = 8. Đây là ngưỡng cấu hình an toàn để khai thác khả năng của GPU mà không gây tràn bộ nhớ.
 - Ngoài ra còn có một số tham số khác nhằm tối ưu quá trình huấn luyện.

3.3 Thiết lập Logic theo dõi và đếm xe

Sau quá trình huấn luyện, đề tài đã trích xuất thành công mô hình tối ưu (best.pt). Mô hình được đưa vào hệ thống để nhận diện, dự đoán hộp giới hạn và phân loại các phương tiện giao thông trong điều kiện thời tiết xấu. Tuy nhiên, để giải quyết bài toán đếm lưu lượng giao thông, hệ thống cần có khả năng theo dõi quỹ đạo chuyển động và thống kê số lượng phương tiện. Toàn bộ quá trình được thiết lập tuần tự thông qua các bước sau:

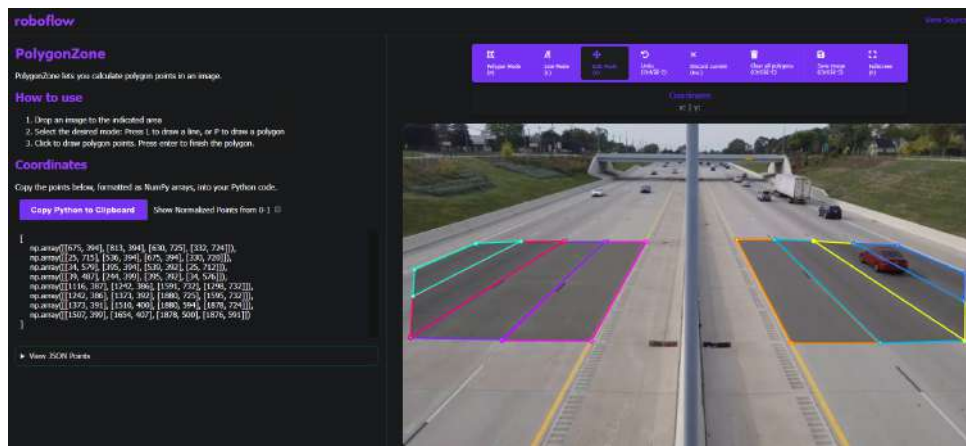
3.3.1 Thiết lập vùng đếm

Hệ thống trong đề tài sử dụng phương pháp đếm dựa trên vùng quan tâm (Region of Interest - RoI).

Region of Interest (RoI) là một vùng không gian được định nghĩa trong khung hình camera, đại diện cho một khu vực thực tế trên đường. Thay vì chỉ sử dụng một vạch kẻ ngang, việc thiết lập các đa giác (polygon) khép kín cho phép hệ thống thống kê chính xác lưu lượng giao thông theo từng làn đường cụ thể, hỗ trợ luồng giao thông phức tạp.

Để có được tập hợp tọa độ các điểm góc của từng làn đường, đề tài ứng dụng công cụ Roboflow Polygon Annotation. Quy trình thiết lập bao gồm:

- Trích xuất một khung hình tĩnh từ video.
- Tải khung hình lên nền tảng Roboflow và sử dụng công cụ vẽ Polygon để khoanh vùng thủ công các làn đường.
- Xuất dữ liệu tọa độ (x,y) từ Roboflow và chuyển đổi thành các mảng **numpy.array** để tích hợp vào mã nguồn hệ thống.



Hình 3.5: Thiết lập vùng đếm.

3.3.2 Thiết lập logic đếm xe

Logic đếm xe được thực hiện qua các bước sau:

- **Trích xuất tọa độ và đặc trưng:** Khi một phương tiện di chuyển, hệ thống ghi nhận mã định danh (ID) từ thuật toán ByteTrack, sử dụng điểm trung tâm ở cạnh dưới của hộp giới hạn làm điểm theo dõi (c_x, c_y) và tịnh tiến điểm c_y lên một khoảng bằng 15% chiều cao hộp giới hạn h . Việc này giúp chuẩn hóa tọa độ cho mọi kích thước phương tiện, đảm bảo điểm theo dõi luôn nằm trong vùng đếm. Tham số chiều cao h cũng được lưu lại để phục vụ cơ chế lọc nhiễu.
- **Kiểm tra vùng đếm:** Hệ thống lưu trữ một tập hợp (Set) chứa ID của các phương tiện đã được đếm. Nếu ID đã tồn tại trong danh sách này, phương tiện sẽ bị bỏ qua ở các khung hình tiếp theo nhằm tránh lỗi đếm lặp. Ngược lại, với các ID chưa có trong tập hợp, hệ thống sử dụng hàm `cv2.pointPolygonTest()` để kiểm tra tọa độ (c_x, c_y) có nằm bên trong vùng đếm hay không. Chỉ khi phương tiện tiến vào bên trong vùng đếm, hệ thống mới tiếp tục thực hiện bước tiếp theo.
- **Cơ chế lọc nhiễu:** Khi phương tiện tiến vào vùng đếm, hệ thống tính toán khoảng cách không gian (Euclidean distance) giữa phương tiện mới và các phương tiện đã đi qua trong khoảng thời gian 20 khung hình. Hệ thống sẽ thiết lập một ngưỡng

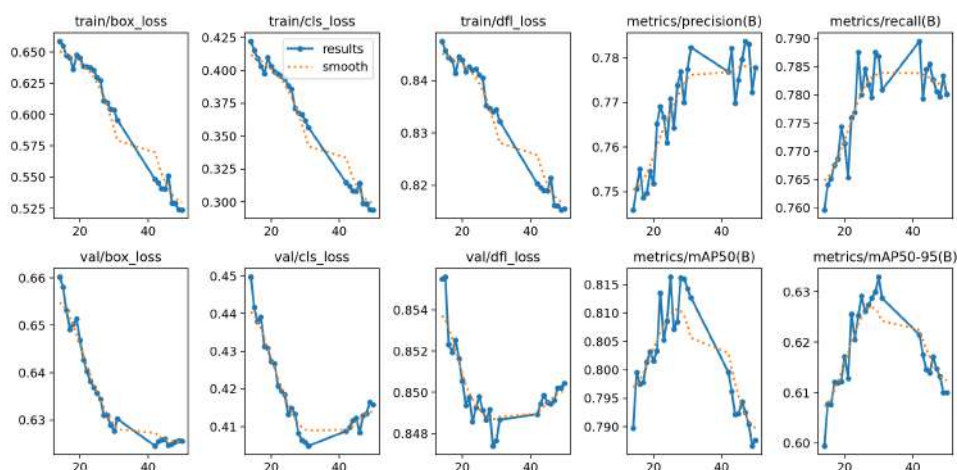
sai lệch tối đa bằng 50% chiều cao h của hộp giới hạn cũ, và được giới hạn trong khoảng từ 30 đến 70 pixel nhằm thích ứng với sự chênh lệch kích thước của phương tiện. Nếu khoảng cách không gian nhỏ hơn ngưỡng giới hạn này, mã định danh mới sẽ được xác định là một ID ảo, từ đó hệ thống bỏ qua việc tăng biến đếm.

- **Xác định hướng và ghi nhận đếm:** Khi phương tiện vượt qua được bộ lọc trên, hệ thống đối chiếu tọa độ trục dọc $c_y(t)$ hiện tại với tọa độ cũ $c_y(t - 1)$ ở khung hình trước để phân tích hướng di chuyển:
 - Nếu $c_y(t) > c_y(t - 1)$ (**phương tiện di chuyển xuống**): Tăng biến đếm đi vào.
 - Nếu $c_y(t) < c_y(t - 1)$ (**phương tiện di chuyển lên**): Tăng biến đếm đi ra.
- Sau khi cập nhật, thông số của phương tiện (tọa độ, chiều cao, frame) tiếp tục được sử dụng làm mốc tham chiếu cho bộ lọc, đồng thời ID bị khóa vào tập hợp để chống đếm lặp. Quá trình này được lặp lại cho toàn bộ các phương tiện trên luồng video.

Chương 4 KẾT QUẢ VÀ ĐÁNH GIÁ

4.1 Kết quả huấn luyện

Sau khi huấn luyện mô hình, ta được kết quả như sau:



Hình 4.1: Kết quả huấn luyện.

4.1.1 Đánh giá hàm mất mát (Loss Curves)

Nhìn vào các biểu đồ `box_loss`, `cls_loss`, `dfl_loss`, ta có thể thấy các sai số trên tập huấn luyện giảm đều từ epoch 1 đến 50. Điều này chứng tỏ kiến trúc mạng đang học và trích xuất đặc trưng hình ảnh tốt. Tuy nhiên, trên tập xác thực, giá trị loss giảm dần và đạt mức cực tiểu ở epoch 30 - 35, sau đó có xu hướng tăng nhẹ. Đây là dấu hiệu cho thấy mô hình bắt đầu xảy ra hiện tượng quá khớp (Overfitting) nếu tiếp tục huấn luyện sâu hơn.

4.1.2 Đánh giá các chỉ số hiệu suất

Độ chính xác (Precision) và Độ bao quát (Recall): Biểu đồ precision và recall đều tăng dần và ổn định ở mức 0.75 - 0.8. Hai chỉ số này cho thấy mô hình đã đạt được sự cân bằng tốt, mô hình dự đoán đúng nhiều hơn và ít bỏ sót đối tượng hơn.

Độ chính xác trung bình ($mAP@50$ và $mAP@50 - 95$): Điều tăng mạnh nhưng sau đó giảm nhẹ xuống quanh mức 0.8 và 0.6. Đây là kết quả chấp nhận được, phản ánh khả năng mô hình có thể hoạt động ổn định trong điều kiện thời tiết khắc nghiệt.

4.1.3 Đánh giá trên tập kiểm thử

Sau khi trích xuất mô hình tối ưu nhất (best.pt), mô hình được đưa vào đánh giá độc lập trên tập Kiểm thử (Test set) bao gồm 6.614 hình ảnh với tổng số 152.772 đối tượng (instances).

Bảng 4.1: Kết quả đánh giá trên tập kiểm thử

Class	Images	Instances	Precision	Recall	$mAP@50$	$mAP@50-95$
All	6.614	152.772	0,735	0,760	0,763	0,588
unidentified	479	1.196	0,449	0,323	0,326	0,212
others	3.415	21.205	0,712	0,881	0,825	0,694
pedestrian	2.685	7.528	0,740	0,687	0,724	0,369
micromobility	1.980	4.217	0,734	0,764	0,726	0,489
car	6.487	105.017	0,926	0,963	0,981	0,823
bus	2.844	4.882	0,889	0,926	0,942	0,825
small truck	3.329	7.083	0,762	0,841	0,862	0,697
truck	1.287	1.644	0,671	0,696	0,721	0,597

Kết quả cho thấy mô hình đạt được độ ổn định cao với chỉ số $mAP@50$ tổng thể đạt 0.763. Trong điều kiện ảnh đầu vào bị nhiễu bởi thời tiết xấu, đây là một kết quả tốt. Cụ thể hơn:

- Mô hình nhận diện tốt với các phương tiện có kích thước hiển thị lớn và cấu trúc rõ ràng. Ví dụ: lớp Ô tô (car) đạt $mAP@50$ lên tới 0.981, lớp Xe buýt (bus) đạt 0.942 và lớp Xe tải nhỏ (small truck) đạt 0.862.
- Các đối tượng kích thước bé như Người đi bộ (pedestrian) và Phương tiện nhỏ (micromobility) đạt điểm $mAP@50$ lần lượt là 0.724 và 0.726. Dù thấp hơn, nhưng

điểm số này phản ánh đúng thực tế khách quan do các đối tượng này chỉ chiếm một số lượng pixel rất nhỏ trên khung hình và dễ bị nhiễu bởi hiện tượng thời tiết.

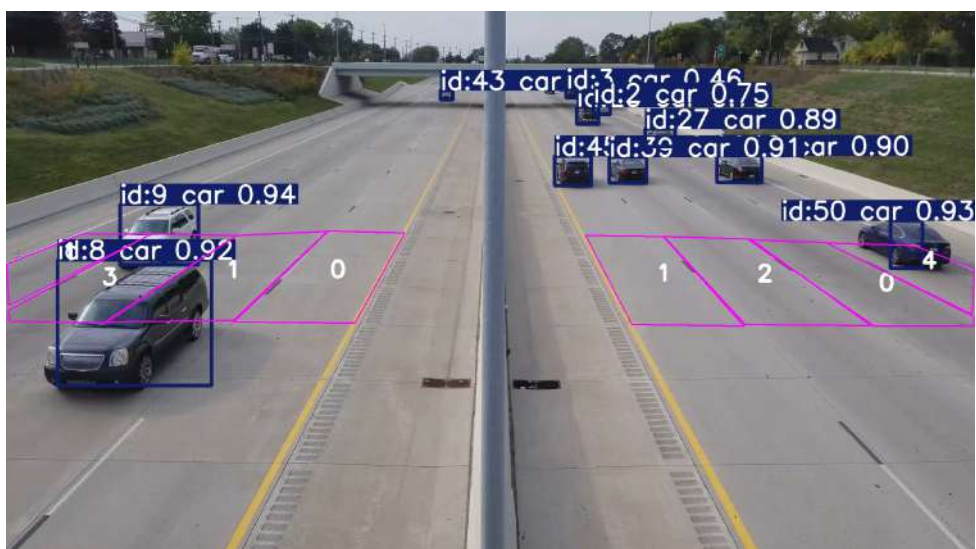
- Lớp Không xác định (unidentified) có điểm số thấp nhất (0.326). Kết quả này hoàn toàn hợp lý vì bản thân các đối tượng này đã thiếu đặc trưng ngay từ khâu gán nhãn thủ công của con người.

4.2 Kết quả thực nghiệm

Sau khi mô hình được huấn luyện, hệ thống được đưa vào chạy thực nghiệm trên các đoạn video giao thông thực tế với 4 kịch bản môi trường điển hình: Điều kiện bình thường (ban ngày), Mưa lớn, Tuyết rơi và Ban đêm.

4.2.1 Đánh giá trong điều kiện thời tiết bình thường

Trong điều kiện ánh sáng tốt, hệ thống hoạt động với độ chính xác gần như tuyệt đối. Mô hình dễ dàng nhận diện phương tiện với độ tự tin rất cao (trên 0.9). Thuật toán ByteTrack duy trì quỹ đạo ổn định, hệ thống đếm hoạt động tốt, không bỏ sót hoặc đếm lặp phương tiện.



Hình 4.2: Kết quả thực nghiệm trong điều kiện thời tiết bình thường.

4.2.2 Đánh giá trong điều kiện thời tiết khắc nghiệt

Kết quả thực nghiệm cho thấy hệ thống vẫn nhận diện và đếm số lượng phương tiện tốt khi điều kiện thời tiết không quá khắc nghiệt (Lượng mưa/tuyết không quá dày đặc, môi trường ban đêm vẫn có ánh sáng đèn đường). Tuy nhiên, với cường độ cực đoan hơn (Mưa lớn, tuyết dày, hoặc ban đêm ở các góc thiếu sáng), mô hình chỉ có thể nhận diện phương tiện với độ tự tin ở mức trung bình hoặc thấp, đôi khi mất dấu hoàn toàn phương tiện, dẫn đến việc đếm không còn chính xác.

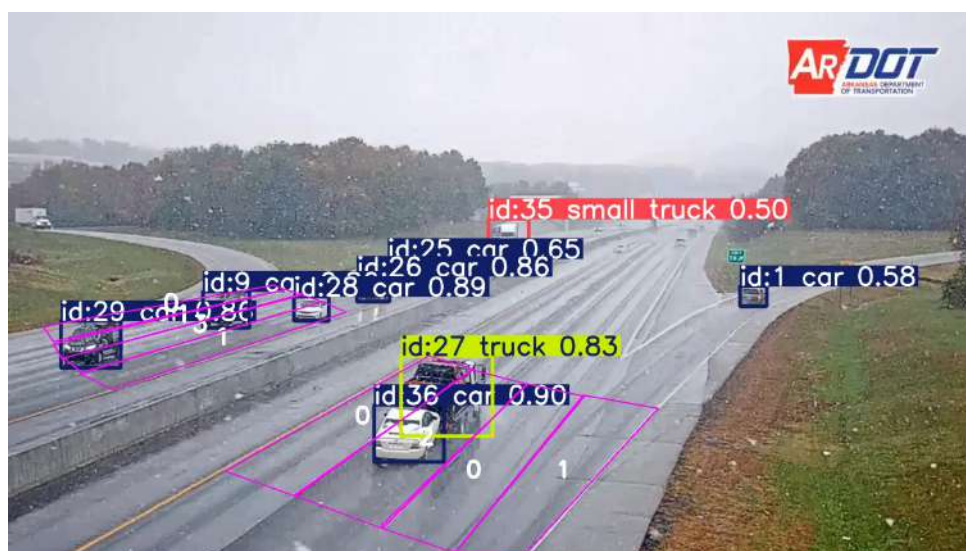
4.2.3 Đánh giá các yếu tố khác

Mặc dù đã áp dụng nhiều kỹ thuật tối ưu từ khâu nhận diện đến logic đếm, hệ thống vẫn tồn tại một số giới hạn nhất định do các rào cản vật lý và bản chất của thuật toán:

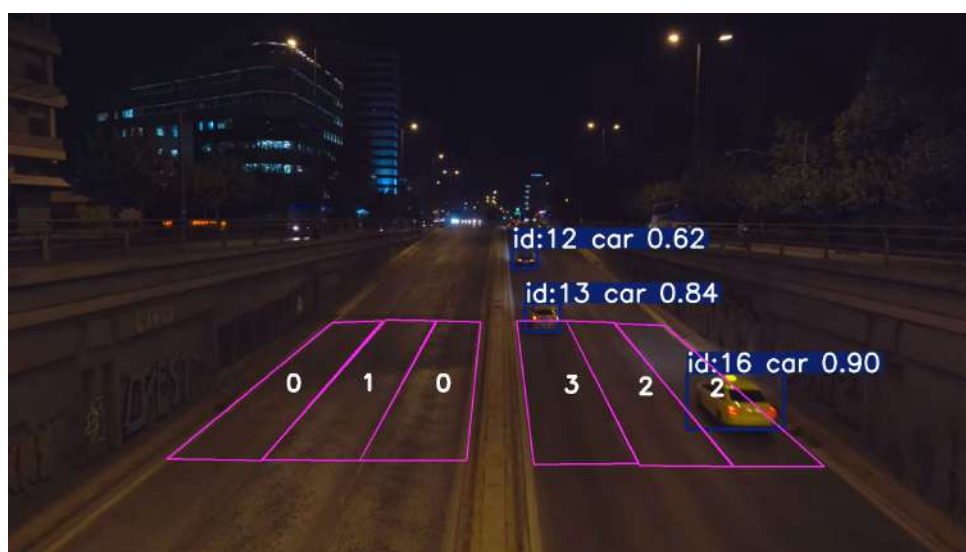
- **Ảnh hưởng của chất lượng video:** Các video chất lượng thấp, mờ nhòe gây khó khăn rất lớn cho quá trình nhận diện xe. Trong hầu hết các trường hợp trên, hệ thống đã không nhận diện đúng hoặc không nhận diện được xe ở khoảng cách trung bình đến xa, làm quá trình đếm trở nên thiếu chính xác.
- **Vị trí thiết lập vùng đếm:** Do phương tiện ở càng xa thì kích thước pixel càng nhỏ, mô hình càng khó nhận diện. Do đó, hệ thống chỉ hoạt động hiệu quả khi vùng đếm được đặt ở khoảng cách gần.
- **Giới hạn của cơ chế lọc nhiễu:** Bộ lọc nhiễu hoạt động tốt trong phần lớn các trường hợp để chống đếm lặp. Tuy nhiên, nếu các phương tiện di chuyển quá sát nhau hoặc di chuyển trong điều kiện thời tiết cực đoan, bộ lọc vẫn có thể xảy ra sai số, gây ra hiện tượng bỏ sót hoặc đếm lặp.



(a) Mưa lớn



(b) Tuyết rơi



(c) Ban đêm

Hình 4.3: Kết quả thực nghiệm trong các điều kiện thời tiết khắc nghiệt

Chương 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đề tài "**Nhận diện và đếm số lượng phương tiện giao thông trong các điều kiện thời tiết khắc nghiệt**" đã giải quyết thành công mục tiêu đề ra bằng việc kết hợp mô hình nhận diện sử dụng mạng nơ-ron học sâu kết với thuật toán theo dõi. Các kết quả đạt được bao gồm:

- Nhận diện tốt các vật thể với nhiều kích thước và phân loại khác nhau.
- Duy trì mã định danh ổn định bằng Bytetrack cho các phương tiện xuyên suốt các khung hình.
- Xây dựng logic đếm xe kết hợp với cơ chế lọc nhiễu, loại bỏ hiện tượng đếm lặp.

5.2 Hướng phát triển trong tương lai

Mặc dù hệ thống đã đạt được độ ổn định cao, bài toán phân tích giao thông đô thị vẫn luôn tồn tại những thách thức cần được tiếp tục nghiên cứu. Trong tương lai, đề tài định hướng mở rộng theo các khía cạnh sau:

- **Tối ưu hóa luồng xử lý trực tiếp:** Hiện tại, hệ thống tách riêng khâu xử lý và khâu xuất kết quả. Vì vậy, hướng phát triển tiếp theo là xây dựng luồng xử lý song song để hệ thống có khả năng đọc dữ liệu trực tiếp từ các luồng camera giao thông IP, thực hiện xử lý và hiển thị kết quả trong khi đảm bảo tốc độ thời gian thực.
- **Tăng cường huấn luyện và tinh chỉnh mô hình:** Tiến hành huấn luyện mô hình với số lượng vòng lặp lớn hơn, kết hợp với các chiến lược giảm tỷ lệ học và tinh chỉnh siêu tham số để mô hình hội tụ tốt hơn, học được các đặc trưng vi mô của

vật thể, từ đó khắc phục tình trạng nhận diện với độ tự tin thấp trong thời tiết cực đoan.

- **Tối ưu hóa logic đếm xe:** Tinh chỉnh lại cơ chế lọc nhiễu, đồng thời tích hợp thêm các yếu tố phân tích động học như vận tốc, hướng vector di chuyển của Bounding Box để bộ lọc có khả năng phân biệt chính xác hơn.
- **Tích hợp mạng tiền xử lý ảnh:** Để cải thiện tình trạng suy giảm chất lượng hình ảnh đầu vào, hệ thống có thể được bổ sung các module tiền xử lý bằng AI (như thuật toán Dehazing khử sương mù, Deraining khử mưa, hoặc Low-light Enhancement tăng cường sáng) để làm sạch khung hình trước khi đưa vào phân tích.

Tài liệu tham khảo

- [1] Xin Lu et al. *MimicDet: Bridging the Gap Between One-Stage and Two-Stage Object Detection*. 2020. arXiv: 2009.11528 [cs.CV]. URL: <https://arxiv.org/abs/2009.11528>.
- [2] Joseph Redmon et al. *You Only Look Once: Unified, Real-Time Object Detection*. 2016. arXiv: 1506.02640 [cs.CV]. URL: <https://arxiv.org/abs/1506.02640>.
- [3] Glenn Jocher et al. “Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models”. In: (2026). DOI: 10.48550/arXiv.2606.03748. URL: <https://arxiv.org/abs/2606.03748>.
- [4] Rahima Khanam and Muhammad Hussain. *YOLOv11: An Overview of the Key Architectural Enhancements*. 2024. arXiv: 2410.17725 [cs.CV]. URL: <https://arxiv.org/abs/2410.17725>.
- [5] Yifu Zhang et al. *ByteTrack: Multi-Object Tracking by Associating Every Detection Box*. 2022. arXiv: 2110.06864 [cs.CV]. URL: <https://arxiv.org/abs/2110.06864>.
- [6] Ngoc Doan-Minh Huynh et al. “TSBOW: Traffic Surveillance Benchmark for Occluded Vehicles Under Various Weather Conditions”. In: *Proceedings of the AAAI Conference on Artificial Intelligence* 40.7 (2026), pp. 5239–5247. DOI: 10.1609/aaai.v40i7.37439. URL: <https://ojs.aaai.org/index.php/AAAI/article/view/37439>.

Phụ lục A Liệt kê source code

Để thuận tiện cho việc đánh giá, toàn bộ mã nguồn của hệ thống đã được đưa lên kho lưu trữ GitHub.

Mã nguồn đầy đủ:

<https://github.com/ToQuangHan25/Traffic-Counting.git>

Dưới đây là trích đoạn các mã nguồn cốt lõi thể hiện đóng góp chính của đề tài.

A.1 A.1 Cấu hình siêu tham số huấn luyện mô hình

Code thiết lập các siêu tham số phục vụ cho quá trình huấn luyện mô hình.

File **train.py**:

```
1 model = YOLO("yolo11m.pt")
2
3 results = model.train(
4     data="/content/datasets/TSBOW/TSBOW.yaml",
5     epochs=50,
6
7     imgsz=1280,
8     batch=8,
9     workers=2,
10
11     patience=10,
12     device="0",
13     project=output_dir,
14     name="tsbow_run",
15     exist_ok=True,
16
17     lr0=0.001,
18     lrf=0.01,
19     freeze=10,
20
21     close_mosaic=10,
22     augment=True
23 )
```

A.2 A.2 Trích xuất tọa độ tham chiếu

Code trích xuất tọa độ điểm theo dõi và chiều cao của phương tiện từ hộp giới hạn.

File **center.py**:

```

1 def get_center(box):
2     cx = int((box[0] + box[2]) / 2)
3     chieu_cao_box = box[3] - box[1]
4     cy = int(box[3] - (chieu_cao_box * 0.15))
5     return (cx, cy, chieu_cao_box)

```

A.3 A.3 Logic đếm phương tiện

Code kiểm tra vùng đếm, lọc nhiễu và thống kê lưu lượng phương tiện.

File `count.py`:

```

1 def count(id_xe, diem_hien_tai, diem_cu, lan_duong, in_counts, out_counts, danh_sach
, frame_hien_tai, chieu_cao_box_hien_tai):
2     cx, cy = diem_hien_tai
3     for i, lan in enumerate(lan_duong):
4         if cv2.pointPolygonTest(lan, diem_hien_tai, False) >= 0:
5             cy_hien_tai = diem_hien_tai[1]
6             cy_cu = diem_cu[1]
7
8             if cy_hien_tai != cy_cu:
9                 is_ghost = False
10                for old_id, (old_cx, old_cy, old_height, old_frame) in
vi_tri_xe_da_dem.items():
11                    frame_lech = frame_hien_tai - old_frame
12
13                    if frame_lech < 20:
14                        khoang_cach = np.sqrt((cx - old_cx)**2 + (cy - old_cy)**2)
15                        ban_kinh_bat_ma = max(30, min(old_height * 0.5, 70))
16
17                        if khoang_cach < ban_kinh_bat_ma:
18                            is_ghost = True
19                            break
20
21                if not is_ghost:
22                    if cy_hien_tai > cy_cu:
23                        in_counts[i] += 1
24                    elif cy_hien_tai < cy_cu:
25                        out_counts[i] += 1
26
27                vi_tri_xe_da_dem[id_xe] = (cx, cy, chieu_cao_box_hien_tai,
frame_hien_tai)
28
29                danh_sach.add(id_xe)
30                break

```